LESSON PLAN · PYTHON (GR 6–8) INTERMEDIATE: PYTHON GAME CODERS · LEVEL 2

Lesson 6: Strings in Depth: Hangman

Module 2 | 60 minutes | slicing, string methods, in | Grades 7–11 · ages 12–16

Objective: Use indexes, slicing, string methods, in and loops over letters to take text apart, and build a Hangman game.

Success criteria

- I can use indexes and slices to get parts of a string.
- I can use string methods like upper, replace, count and split.
- I can loop over the letters of a word and use in to search text.
- I can build a new string piece by piece.

Key vocabulary

- **index**: The position number of a letter, starting at 0.
- **slice**: A piece of a string, like word[0:3].
- **method**: A command attached to a value with a dot, like word.upper().
- **immutable**: Cannot be changed; strings are immutable, so we build new ones.
- **palindrome**: A word that reads the same backwards, like Malayalam.
- **.split()**: Breaks a sentence into a list of words.

You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Project files: word_explorer.py

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game: Human Hangman**	Play Human Hangman (unplugged) with RANGOLI. Ask what the “program” checked and redrew for each guess.
5 min	**Hook and video**	Show the hook. Chat poll: “What is monsoon backwards?” Play the video with captions; pause at “Index and slice” to predict city[2:6].
10 min	**Live demo and build-along**	Run the live demo with the index strip, then everyone types the word explorer. Thumbs check: “Is Malayalam a palindrome?”
8 min	**Challenge, predict and match**	Vowel counter and palindrome challenge, the slice prediction and the monsoon match. Quick competition: who finds the longest palindrome in English or Hindi?

15 min	**Hangman in breakout pairs**	Breakout pairs build Hangman: one writes show_word and tests it, the other writes the game loop; then combine. Partners play with each other's word lists.
6 min	**Debug race**	Race to fix "the hashtag maker". The winner explains why brackets matter.
7 min	**Hangman show-and-tell**	Two learners run Hangman live while the class guesses letters in the chat. Cheer every correct letter.
4 min	**Wrap-up and exit ticket**	Fist-to-five on the success criteria. Exit ticket: Kabaddi slices.

Unplugged warm-up

Human Hangman (5 min game): the teacher draws blanks for a word like R A N G O L I and learners guess letters in the chat. Keep a lives counter on the board. Afterwards ask: what did I check for every guess, and what did I redraw each time? (Is the letter in the word? Redraw the blanks with the found letters.) That is the program.

Teaching the idea

A string is a sequence of characters, each with an index from 0 (and negative indexes from the end). Slicing [start:stop:step] cuts out a piece and stops before stop; [::-1] reverses. String methods like upper, lower, title, replace, count and split return NEW strings, because strings cannot be changed in place. in searches inside a string, and for letter in word loops over characters. Hangman uses all of these: in to check a guess, a loop to build the display, and a new string each time.

Analogy: A string is like a train of carriages with numbers painted on them, starting at 0. A slice is uncoupling a group of carriages, and reversing is driving the train backwards. You cannot repaint one carriage, but you can build a new train.

Live demo: 1) Type city = "Bengaluru" and write the indexes 0 to 8 under it on the whiteboard. Print city[0], city[-1] and city[0:5]. 2) Ask the class to predict city[2:6] before running. 3) Print city[::-1]. 4) Show text.title(), replace, count and split on "chai and samosa". 5) Try word[0] = "M" and read the TypeError; fix with "M" + word[1:]. 6) Build the show_word loop for "mango" with guessed = ["m", "o"] and print it.

Questions to ask

Ask	Listen for
What is word[-1]?	The last letter of the word.
What does "Bengaluru"[0:5] give?	Benga, the letters at index 0 to 4.
Why does word[0] = "M" crash?	Strings cannot be changed; build a new string instead.

How can you check if a letter is in the secret word?	if guess in secret:
Why do methods need ( ) at the end?	The brackets run the method; without them you just name it.

Differentiation

Support
Give a printed index strip for “monsoon” and a fill-in show_word skeleton. Learners can start Hangman with a fixed secret = "mango" and add random.choice last.

Stretch
Draw the hangman picture with a list of ASCII art stages, or give a hint letter when only 2 lives are left.

Common misconceptions

Misconception	How to fix it
Indexes start at 1	The first letter is at index 0.
word[0:3] includes index 3	A slice stops just before the end number.
word.upper() changes word	It returns a new string; store it: word = word.upper().
text.split gives the words	Methods need brackets: text.split().

Assessment

Use the Python Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
name = "Kabaddi". What do name[0:3] and name[::-1] give?

Homework
Play Hangman with someone at home using words they choose for your list. Add a hint: print the first letter of the word when lives reaches 2.

Answer key — Kitabash activities

1. Python Lab: vowel counter and palindrome checker (perform · python lab)

Goals checked: Counts vowels · Spots palindromes · Loops over letters

One solution:

```
def count_vowels(word):
    count = 0
    for letter in word.lower():
        if letter in "aeiou":
            count += 1
    return count

def is_palindrome(word):
    w = word.lower()
    return w == w[::-1]

print(count_vowels("Kitabash"))
print(is_palindrome("Malayalam"))
```

2. Predict the output: slice and lower (perform · code output)

3. Match each expression to its result when s = "monsoon" (perform · matching)

s[0] → "m" · s[-1] → "n" · s[1:4] → "ons" · s[::-1] → "noosnom" · len(s) → 7 · s.upper() → "MONSOON"

4. Project: Hangman (perform · python lab)

Goals checked: show_word shows the gaps · Six wrong guesses lose all lives · Reveals the word at the end · Uses random, a loop and input

Example program:

```
# Hangman
# Made by: Riya
import random

words = ["cricket", "monsoon", "rangoli", "python", "mango", "kabaddi"]

def show_word(word, guessed):
    shown = ""
    for letter in word:
        if letter in guessed:
            shown += letter + " "
        else:
            shown += "_ "
    return shown.strip()

secret = random.choice(words)
guessed = []
lives = 6
print("=== HANGMAN ===")
print(f"The word has {len(secret)} letters. You have {lives} lives.")

while lives > 0 and "_" in show_word(secret, guessed):
    print(show_word(secret, guessed))
    guess = input("Guess a letter: ").strip().lower()
    if len(guess) != 1 or not guess.isalpha():
        print("Please type one letter.")
    elif guess in guessed:
        print("You already tried that letter.")
    elif guess in secret:
        guessed.append(guess)
        print("Yes! It is in the word.")
    else:
        guessed.append(guess)
        lives -= 1
        print(f"No. Lives left: {lives}")

if lives > 0:
    print(f"You win! The word was {secret}.")
else:
    print(f"Out of lives! The word was {secret}.")
```

5. Debug it: the hashtag maker (probe · python lab)

Goals checked: Makes #DiwaliSaleToday · Works for any phrase

One solution:

```
text = input("Phrase: ")
words = text.split()
tag = "#"
for w in words:
    tag = tag + w.title()
print(tag)
```

Answer key — worksheet

1. **Key words:** index = The position number of a letter, starting at 0. · slice = A piece of a string, like word[0:3]. · method = A command attached to a value with a dot, like word.upper(). · immutable = Cannot be changed; strings are immutable, so we build new ones.
2. **Plan your program:** Goals: Counts vowels; Spots palindromes; Loops over letters. One solution: def count_vowels(word): count = 0 for letter in word.lower(): if letter in "aeiou": count += 1 return count def is_palindrome(word): w = word.lower() return w == w[::-1] print(count_vowels("Kitabash")) print(is_palindrome("Malayalam"))
3. **Predict the output: slice and lower:** A. diwl
4. **Match each expression to its result when s = "monsoon":** s[0] → "m" · s[-1] → "n" · s[1:4] → "ons" · s[::-1] → "noosnom" · len(s) → 7 · s.upper() → "MONSOON"
5. **Plan your program:** Goals: Makes #DiwaliSaleToday; Works for any phrase. One solution: text = input("Phrase: ") words = text.split() tag = "#" for w in words: tag = tag + w.title() print(tag)

Kitabash original curriculum.